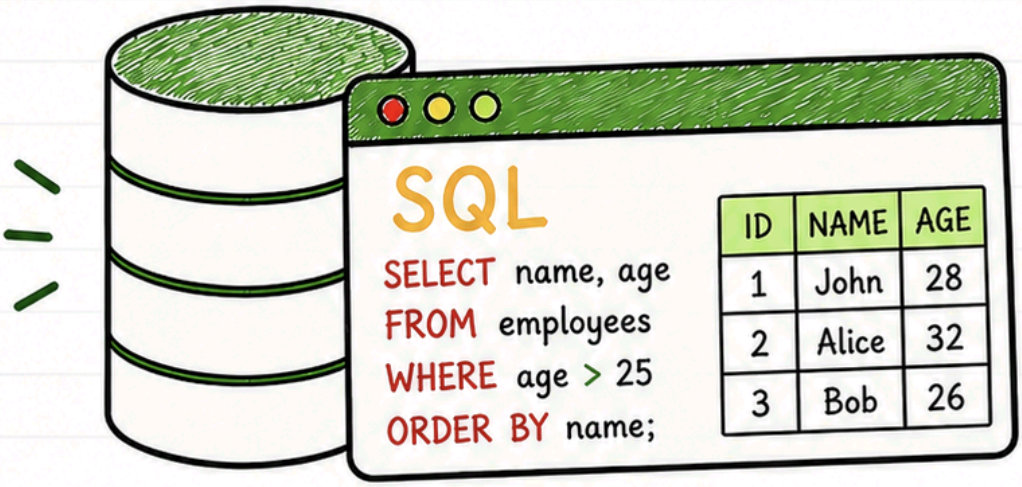
@qa_insights

SQL

COMPLETE NOTES

(Structured Query Language)



BEGINNER TO ADVANCED

SWIPE TO NEXT →

FOLLOW | SHARE | COMMENT | LIKE

@qa_insights | Quality Insights, Better Testing!

@qa_insights

1

What is SQL?

SQL (Structured Query Language) is the standard language used to communicate with relational databases to store, retrieve, update and manage data efficiently.

1 Why Databases?

- ✓ Store large amounts of data in an organized way.
- ✓ Ensure data accuracy, consistency and security.
- ✓ Enable fast retrieval and analysis of data.
- ✓ Support multiple users and real-time applications.

2. DBMS vs RDBMS

DBMS 	RDBMS 
• Stores data as files or simple structures. • Relationships are limited. • Lower data integrity. • No support for relationships using keys. Examples: File System, XML	• Stores data in tables. • Supports relationships using Primary & Foreign Keys. • Enforces constraints. • Better consistency and integrity. Examples: MySQL, PostgreSQL, SQL Server, Oracle

3 SQL Use Cases


- Web Applications


- Banking & Finance


- E-commerce


- Data Analysis & Reporting


- QA Automation (Test Data)


- DevOps & Monitoring

4. Popular Databases

 MySQL	• Open-source relational database, widely used in web applications.
 PostgreSQL	• Advanced open-source database known for reliability and performance.
 Microsoft® SQL Server	• Microsoft's relational database for enterprise applications.
ORACLE	• Enterprise-grade database used in large scale and mission-critical systems.


Interview Fact | SQL is pronounced either "S-Q-L" or "Sequel". Both pronunciations are widely accepted.


Pro Tip | SQL is a declarative language. You tell the database WHAT data you want, not HOW to get it.

@qa_insights

2

Database Basics

A database organizes data so it can be easily stored, retrieved, updated and managed efficiently.

1. Core Terms

- **Database :** A collection of related data.
- **Table :** A set of data organized in rows and columns.
- **Row :** Each record in a table.
- **Column :** Each field or attribute in a table.

2. Table Example

Table: Employees

Column →

EmpID (PK)	Name	Department	Salary	JoinDate
101	John	IT	60000	2023-01-15
102	Alice	HR	55000	2023-02-20
103	Bob	Finance	70000	2023-03-10

Row →

Primary Key
Unique identifier for each row.

Data
Actual values stored in the table.

3. Keys

Primary Key (PK) :
- Uniquely identifies each row in a table.
- Cannot contain NULL values.

Foreign Key (FK) :
- Refers to the Primary Key of another table.
- Maintains relationship between tables.

Departments

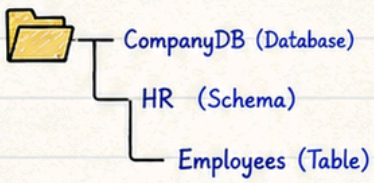
DeptID (PK)	DeptName
1	IT
2	HR
3	Finance

DeptID is Foreign Key →

Employees

EmpID (PK)	Name	DeptID (FK)
101	John	1
102	Alice	2
103	Bob	3

4. Schema



5. Common Data Types

Data Type	Description
INT	Stores whole numbers. e.g., 101, -5, 1000
VARCHAR(n)	Stores variable length strings up to n characters. e.g., VARCHAR(50)
DATE	Stores date values. e.g., 2024-05-20
BOOLEAN	Stores true or false values. e.g., TRUE, FALSE
DECIMAL(p,s)	Stores decimal numbers. p = precision, s = scale. e.g., DECIMAL(10,2) → 12345.67

Pro Tip | Choose the right data type to save space, improve performance and ensure data accuracy.

Interview Fact | A well-designed database with proper keys and relationships ensures data integrity and efficiency.

@qa_insights

3

SELECT Statement

SELECT is used to retrieve data from one or more tables in a database.

1 Basic SELECT

✓ Retrieves all rows and columns from a table.

```
SELECT *
FROM Employees;
```

2 Select Multiple Columns

✓ Retrieve specific columns instead of all.

```
SELECT EmpID, Name, Salary
FROM Employees;
```

3 DISTINCT

✓ Returns only unique values.

```
SELECT DISTINCT Department
FROM Employees;
```

4 LIMIT / TOP

✓ LIMIT is used in MySQL, PostgreSQL, SQLite.

```
SELECT Name, Salary
FROM Employees
LIMIT 5;
```

✓ TOP is used in SQL Server.

```
SELECT TOP 5 Name, Salary
FROM Employees;
```

Note: Database-specific syntax may vary.
Use LIMIT or TOP to restrict the number of rows.

5 Aliases (AS)

✓ Aliases give a temporary name to column or table.

```
SELECT Name AS EmployeeName,
       Salary AS MonthlySalary
FROM Employees;
```

→ Improves readability

6 SELECT * Best Practices

- ✓ Avoid using SELECT * in production.
- ✓ It retrieves unnecessary columns.
- ✓ Increases network traffic and memory usage.
- ✓ It may break queries if new columns are added.
- ✓ Always specify only the columns you need.

Bad ✗

```
SELECT *
FROM Employees;
```

- ✗ Returns all columns
- ✗ Consumes more resources
- ✗ Not future-proof

Good ✓

```
SELECT EmpID, Name, Salary
FROM Employees;
```

- ✓ Only required columns
- ✓ Better performance
- ✓ More secure & maintainable

Pro Tip | Always select only what you need for better performance and clarity.

Interview Fact | SELECT is one of the most frequently asked topics in SQL interviews.

4

Filtering Data (WHERE Clause)

The WHERE clause is used to filter rows based on a condition.

1 Comparison & Logical Operators

- ✓ AND : Returns true if both conditions are true.
- ✓ OR : Returns true if any one condition is true.
- ✓ NOT : Reverses the result.

Example

	Description	SQL Example
AND	Salary > 50000 AND Dept = 'IT'	SELECT * FROM Employees WHERE Salary > 50000 AND Dept = 'IT';
OR	Dept = 'IT' OR Dept = 'HR'	SELECT * FROM Employees WHERE Dept = 'IT' OR Dept = 'HR';
NOT	NOT Dept = 'IT'	SELECT * FROM Employees WHERE NOT Dept = 'IT';

2. Other Filtering Conditions

BETWEEN

Select values within a range (inclusive).

```
SELECT * FROM Employees
WHERE Salary BETWEEN
40000 AND 80000;
```

IN

Match any value in a list.

```
SELECT * FROM Employees
WHERE Dept IN
('IT', 'HR', 'Finance');
```

LIKE

Search for a pattern in text.

```
SELECT * FROM Employees
WHERE Name LIKE 'A%';
```

Wildcards

% → any number of characters

_ → single character

Examples:

- 'A%' → starts with A
- '%son' → ends with son
- '_n_' → any 3-letter word with n in middle

IS NULL

Check for empty / missing values.

```
SELECT * FROM Employees
WHERE ManagerID IS NULL;
```

IS NOT NULL

Check for non-empty / existing values.

```
SELECT * FROM Employees
WHERE ManagerID IS NOT NULL;
```

Note

- Comparisons with NULL using = or <> do not work. Use IS NULL / IS NOT NULL.
- String comparisons may be case-sensitive depending on the database.

3 Quick Reference

AND	OR	NOT	BETWEEN	IN	LIKE
Both conditions must be true.	Any one condition can be true.	Reverse the result.	10 20 30 40 Between range (inclusive).	Match any value in the list.	A% Pattern search using wildcards.

Pro Tip | Combine conditions smartly to get exact data you need.

Interview Fact | WHERE clause is applied before GROUP BY and ORDER BY.

@qa_insights

5

Sorting & Limiting

These clauses help you sort the result and control the number of rows returned.

1 ORDER BY

✓ Sorts the result set based on one or more columns.

```
SELECT EmpID, Name, Salary
FROM Employees
ORDER BY Salary; -- Default ASC
```

★ ORDER BY is always used at the end of the query.

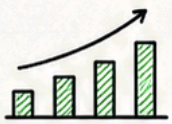
2 ASC vs DESC

✓ ASC = Ascending (Smallest to Largest)

✓ DESC = Descending (Largest to Smallest)

ASC (Default)

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary ASC;
```



DESC

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary DESC;
```

3 LIMIT Clause

✓ Limits the number of rows returned.

```
-- MySQL, PostgreSQL, SQLite
SELECT Name, Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 5;
```

★ Returns only the first 5 rows.

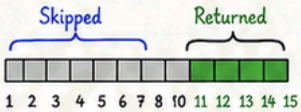
4 OFFSET Clause

✓ Skips a specified number of rows.

✓ Used with LIMIT.

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 5 OFFSET 10;
```

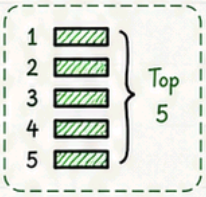
- Skips first 10 rows
- Then returns next 5 rows



5 TOP (SQL Server)

✓ Specifies the number of rows to return right after SELECT.

```
SELECT TOP 5 Name, Salary
FROM Employees
ORDER BY Salary DESC;
```



6 FETCH FIRST (ANSI SQL Standard)

✓ Standard way to limit rows in modern SQL.

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary DESC
FETCH FIRST 5 ROWS ONLY;
```

- Alternative to LIMIT / TOP
- Supported in: DB2, Oracle, PostgreSQL (v13+), SQL Server (with OFFSET)...

With OFFSET

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary DESC
OFFSET 10 ROWS
FETCH FIRST 5 ROWS ONLY;
```

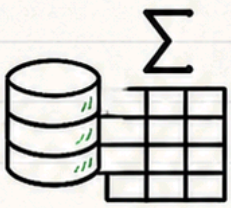
- Skips 10 rows
- Then returns next 5 rows

Pro Tip | Always use ORDER BY with LIMIT / TOP / FETCH for consistent results.

Interview Fact | Without ORDER BY, the order of rows is not guaranteed. Always specify the sorting column.

6

Aggregate Functions



Aggregate functions perform calculations on a set of values and return a single result.

1 Aggregate Functions Overview

COUNT()	SUM()	AVG()	MIN()	MAX()
Counts the number of rows. COUNT(column) → counts non-NULL values.	Returns the total sum of numeric values.	Returns the average of numeric values.	Returns the smallest value.	Returns the largest value.

2 Practical Examples

Employees

EmpID	Name	Department	Salary
101	John	IT	60000
102	Alice	HR	55000
103	Bob	Finance	70000
104	Charlie	IT	45000
105	David	Finance	65000

```
SELECT COUNT(*)
FROM Employees;
```
Result: 5

```
SELECT COUNT(Department)
FROM Employees;
```
Result: 5

```
SELECT SUM(Salary)
FROM Employees;
```
Result: 295000

```
SELECT AVG(Salary)
FROM Employees;
```
Result: 59000.00

```
SELECT MIN(Salary)
FROM Employees;
```
Result: 45000

```
SELECT MAX(Salary)
FROM Employees;
```
Result: 70000

3 Notes & Tips

- ✓ COUNT(*) counts all rows including NULL values.
- ✓ COUNT(column) counts only non-NULL values.
- ✓ SUM(), AVG(), MIN(), MAX() ignore NULL values.
- ✓ Use aggregate functions with GROUP BY to get results per group.

Example with GROUP BY

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department;
```

Department	AvgSalary
IT	52500.00
HR	55000.00
Finance	67500.00

Pro Tip — Aggregate functions are powerful with GROUP BY, HAVING and JOINs for real-world analytics.

Interview Fact — Aggregate functions are commonly asked in SQL interviews for reporting and analytics queries.

GROUP BY & HAVING

GROUP BY is used to group rows that have the same values into summary rows.
HAVING is used to filter groups (aggregated results).

1 GROUP BY

Groups rows based on one or more columns and is usually used with aggregate functions.

Syntax

```
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1, column2, ...
ORDER BY column1;
```

Common Aggregate Functions

COUNT()	– counts rows
SUM()	– total of values
AVG()	– average of values
MIN()	– minimum value
MAX()	– maximum value

Note

Every column in SELECT must be either:

- in GROUP BY, or
- used inside an aggregate function.

2 HAVING

Filters the groups formed by GROUP BY. It works on aggregate values and cannot be used without GROUP BY.

Syntax

```
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1
HAVING aggregate_condition
ORDER BY column1;
```

Key Difference

WHERE	HAVING
• Filters rows before grouping	• Filters groups after grouping
• Used with individual rows	• Used with aggregate values
• No aggregate functions	• Works with aggregate functions

Interview Tip

Use WHERE to reduce rows early.
Use HAVING to filter aggregated results.

3 Grouping Rules

- ✓ All non-aggregate columns in SELECT must appear in GROUP BY.
- ✓ You cannot use aggregate functions in WHERE.
- ✓ HAVING is used to filter groups.
- ✓ Column aliases cannot be used in HAVING (use the full expression).

Valid Example

```
SELECT DeptID, COUNT(*) AS EmpCount
FROM Employees
GROUP BY DeptID
HAVING COUNT(*) > 3
ORDER BY EmpCount DESC;
```

Invalid Example

```
SELECT DeptID, Name, COUNT(*)
FROM Employees
GROUP BY DeptID;
```

↳ ✗ Name is not in GROUP BY and not inside aggregate.

4 Interview Examples

1. Count employees in each department

```
SELECT DeptID, COUNT(*) AS EmpCount
FROM Employees
GROUP BY DeptID
ORDER BY EmpCount DESC;
```

DeptID	EmpCount
10	5
20	4
30	2

Counts employees in each department.

2. Departments with more than 3 employees

```
SELECT DeptID, COUNT(*) AS EmpCount
FROM Employees
GROUP BY DeptID
HAVING COUNT(*) > 3
ORDER BY EmpCount DESC;
```

DeptID	EmpCount
10	5
20	4

Filters groups having more than 3 employees.

3. Total salary > 60000 per department

```
SELECT DeptID, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY DeptID
HAVING SUM(Salary) > 60000
ORDER BY TotalSalary DESC;
```

DeptID	TotalSalary
20	95000
10	72000

Filters departments with total salary greater than 60000.

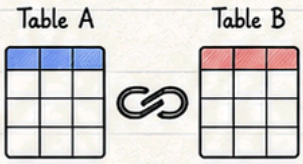
Pro Tip

- ✓ Apply WHERE before GROUP BY to reduce data early.
- ✓ Use HAVING for conditions on aggregates only.
- ✓ Always ORDER BY aggregated results for better readability.

Remember

GROUP BY groups data
HAVING filters groups
WHERE filters rows

SQL Joins



Joins are used to combine rows from two or more tables based on a related column between them.

1 Sample Tables

Table A (Employees)

EmpID	EmpName	DeptID
1	John	10
2	Alice	20
3	Bob	30
4	David	NULL

Table B (Departments)

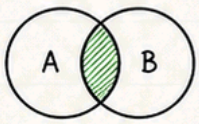
DeptID	DeptName
10	IT
20	HR
30	Finance
40	Marketing

Key Point

We join Table A and Table B using: A.DeptID = B.DeptID (Common column)

2 Types of SQL Joins

INNER JOIN

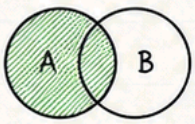


Returns only matching rows from both tables.

```
SELECT *
FROM A
INNER JOIN B
ON A.DeptID = B.DeptID;
```

Result: Only common rows

LEFT JOIN

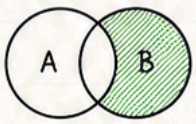


Returns all rows from left table (A) and matched rows from right table (B).

```
SELECT *
FROM A
LEFT JOIN B
ON A.DeptID = B.DeptID;
```

Result: All from A + matched from B

RIGHT JOIN

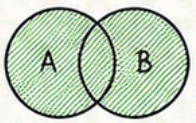


Returns all rows from right table (B) and matched rows from left table (A).

```
SELECT *
FROM A
RIGHT JOIN B
ON A.DeptID = B.DeptID;
```

Result: All from B + matched from A

FULL OUTER JOIN

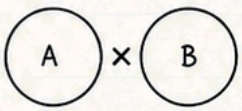


Returns all rows from both tables. Matched rows where available.

```
SELECT *
FROM A
FULL OUTER JOIN B
ON A.DeptID = B.DeptID;
```

Result: All from A and All from B

CROSS JOIN

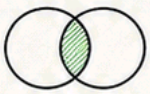
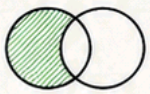
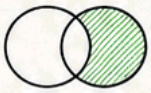
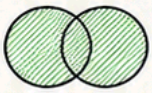
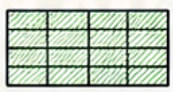


Returns the Cartesian product (each row of A with every row of B).

```
SELECT *
FROM A
CROSS JOIN B;
```

Result: Rows = A count × B count

3 Visual Summary

INNER JOIN	LEFT JOIN	RIGHT JOIN	FULL OUTER JOIN	CROSS JOIN
				
Only Matches	All Left + Matches	All Right + Matches	All Left + All Right	Cartesian Product

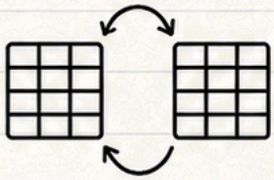
Pro Tip

- ✓ Always specify the join condition using ON.
- ✓ Use table aliases for better readability.
- ✓ Understand the data first, then choose the right join.

Interview Fact

Joins are one of the most important SQL concepts and frequently asked in interviews.

Self Join & UNION



Self Join is a regular join where a table is joined with itself. UNION operators are used to combine results from two or more SELECT statements.

1 SELF JOIN

Used to compare rows within the same table.

Employees

EmpID	Name	ManagerID
1	Alice	NULL
2	Bob	1
3	Charlie	1
4	David	2
5	Eve	2

```
SELECT E.Name AS Employee,
       M.Name AS Manager
FROM Employees E
LEFT JOIN Employees M
ON E.ManagerID = M.EmpID;
```

Result

Employee	Manager
Alice	NULL
Bob	Alice
Charlie	Alice
David	Bob
Eve	Bob

Key Point

- ✓ Alias (E, M) is mandatory in Self Join.
- ✓ Useful for hierarchical data (Manager–Employee).

2 SET OPERATORS

Used to combine results of two or more SELECT statements.

Table A

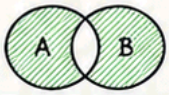
ID	Value
1	A
2	B
3	C

Table B

ID	Value
3	C
4	D
5	E

UNION

Returns unique rows from both queries. Duplicates removed.



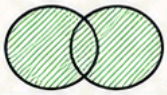

```
SELECT ID, Value FROM A
UNION
SELECT ID, Value FROM B;
```

Result:

ID	Value
1	A
2	B
3	C
4	D
5	E

UNION ALL

Returns all rows from both queries. Duplicates included.



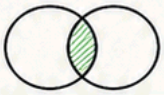
```
SELECT ID, Value FROM A
UNION ALL
SELECT ID, Value FROM B;
```

Result:

ID	Value
1	A
2	B
3	C
3	C
4	D
5	E

INTERSECT

Returns only the common rows.



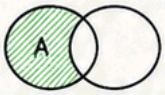
```
SELECT ID, Value FROM A
INTERSECT
SELECT ID, Value FROM B;
```

Result:

ID	Value
3	C

EXCEPT / MINUS

Returns rows from A that are not in B.




```
SELECT ID, Value FROM A
EXCEPT
SELECT ID, Value FROM B;
(In Oracle use MINUS)
```

Result:

ID	Value
1	A
2	B

3 Quick Comparison

Operator	Removes Duplicates?	Keeps Duplicates?	Common Rows Only?	Left Over (A–B)?	Order of Columns Must Match?
UNION	✓	✗	✗	✗	✓
UNION ALL	✗	✓	✗	✗	✓
INTERSECT	✓	✗	✓	✗	✓
EXCEPT / MINUS	✓	✗	✗	✓	✓

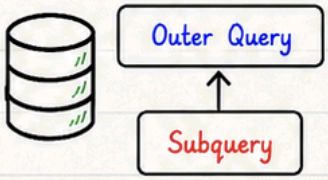
Notes

- All SELECT statements must have the same number of columns with compatible data types.
- ORDER BY can be used only once at the end of the final result.

Pro Tip | Use UNION ALL instead of UNION when duplicates are allowed for better performance.

Interview Fact | Set operators are commonly asked in SQL interviews to test your understanding of result set operations.

Subqueries



A subquery (inner query) is a query inside another SQL statement. The inner query runs first.

1 Single-Row Subquery

Returns exactly one row (one value).
Use with =, <, >, <=, >=, <> operators.

```
SELECT Name, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary)
                FROM Employees);
```

Result:
Employees having salary greater than the average salary.

2 Multi-Row Subquery

Returns multiple rows (multiple values).
Use with IN, NOT IN, ANY, ALL operators.

```
SELECT Name
FROM Employees
WHERE DeptID IN (SELECT DeptID
                 FROM Departments
                 WHERE Location = 'Hyderabad');
```

Result:
Employees who belong to departments in Hyderabad.

3 Correlated Subquery

The subquery refers to columns from the outer query.
It is executed once for each row of the outer query.

```
SELECT E1.Name, E1.Salary
FROM Employees E1
WHERE E1.Salary > (SELECT AVG(E2.Salary)
                   FROM Employees E2
                   WHERE E2.DeptID = E1.DeptID);
```

Result:
Employees who earn more than the avg salary in their own department.

4 EXISTS & NOT EXISTS

Often better than IN/NOT IN, especially with NULLs.

EXISTS

```
SELECT Name
FROM Employees E
WHERE EXISTS (SELECT 1
              FROM Departments D
              WHERE D.DeptID = E.DeptID);
```

Result:
Employees for whom a matching department exists.

NOT EXISTS

```
SELECT Name
FROM Employees E
WHERE NOT EXISTS (SELECT 1
                  FROM Departments D
                  WHERE D.DeptID = E.DeptID);
```

Result:
Employees who do NOT have a matching department.

5 IN vs EXISTS

Aspect	IN	EXISTS	Example
How it works	Compares value with a list	Checks existence of rows	`SELECT Name FROM Employees WHERE DeptID IN (SELECT DeptID FROM Departments);`	`SELECT Name FROM Employees E WHERE EXISTS (SELECT 1 FROM Departments D WHERE D.DeptID = E.DeptID);`
NULL handling	If subquery returns NULL, result may be unexpected.	NOT affected by NULLs.	
Performance	Good for smaller result sets.	Often better for large result sets.	
Readability	Simple and easy.	More explicit intent.	
Use when	When you need to compare a value with a list.	When you only need to check if matching rows exist.	

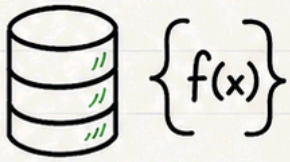
Key Tips

- ✓ Use single-row subquery with comparison operators (=, <, > ...).
- ✓ Use multi-row subquery with IN, NOT IN, ANY, ALL.
- ✓ Prefer EXISTS over IN when dealing with NULLs.
- ✓ Correlated subqueries can be slower; use JOINs when possible.

Pro Tip — Always test subqueries separately first. Then use them inside the outer query.

Interview Fact — Subqueries are a favorite SQL interview topic. Practice them well with different datasets and understand when to use each type.

SQL Functions



SQL Functions are pre-defined operations that perform calculations and return a single value.

1 String Functions

Used to manipulate and extract data from strings.

Function	Description	Example	Example Output
UPPER(str)	Converts string to upper case.	SELECT UPPER('sql insights') AS UpperText;	SQL INSIGHTS
LOWER(str)	Converts string to lower case.	SELECT LOWER('SQL INSIGHTS') AS LowerText;	sql insights
LENGTH(str)	Returns the number of characters in a string.	SELECT LENGTH('SQL Insights') AS Len;	12
SUBSTRING(str, start, len)	Extracts part of a string. (start is 1-based index)	SELECT SUBSTRING('SQL Insights', 5, 8) AS SubStr;	Insights

2 Numeric Functions

Used to perform mathematical operations.

Function	Description	Example	Example Output
ROUND(value, precision)	Rounds a number to the specified decimal places.	SELECT ROUND(123.4567, 2) AS Rounded;	123.46
CEIL(value)	Returns the smallest integer greater than or equal to value.	SELECT CEIL(45.23) AS CeilValue;	46

3 Date Functions

Used to work with dates and times.

Function	Description	Example	Example Output
NOW()	Returns current date and time.	SELECT NOW() AS CurrentDateTime;	2025-05-13 14:30:25
CURRENT_DATE	Returns current date (YYYY-MM-DD).	SELECT CURRENT_DATE AS Today;	2025-05-13
DATE_ADD(date, INTERVAL value unit)	Adds a time interval to a date.	SELECT DATE_ADD('2025-05-13', INTERVAL 7 DAY) AS NextWeek;	2025-05-20

Pro Tip

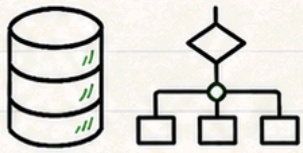
- ✓ Functions can be used in SELECT, WHERE, ORDER BY, HAVING clauses.
- ✓ Most functions can be nested.
- ✓ Always check DBMS-specific syntax variations for date functions.

Interview Fact

Functions help in data formatting, calculations and making queries dynamic and powerful.

12

CASE Statement



CASE statement is used to perform conditional logic and return values based on conditions.

1 CASE WHEN (Simple CASE)

Compares an expression with multiple values.

```
SELECT Name, Salary,
  CASE Department
    WHEN 'IT' THEN 'Technology'
    WHEN 'HR' THEN 'Human Resources'
    WHEN 'Finance' THEN 'Financial'
    ELSE 'Other'
  END AS DeptCategory
FROM Employees;
```

Result:

Name	Salary	DeptCategory
Alice	60000	Technology
Bob	55000	Human Resources
Charlie	70000	Financial
David	45000	Technology
Eve	65000	Other

2 CASE WHEN (Searched CASE)

Evaluates Boolean conditions.

```
SELECT Name, Salary,
  CASE
    WHEN Salary >= 70000 THEN 'High'
    WHEN Salary BETWEEN 50000 AND 69999
      THEN 'Medium'
    ELSE 'Low'
  END AS SalaryLevel
FROM Employees;
```

Result:

Name	Salary	SalaryLevel
Alice	60000	Medium
Bob	55000	Medium
Charlie	70000	High
David	45000	Low
Eve	65000	Medium

3 IFNULL() / COALESCE()

Used to handle NULL values and provide a default value.

IFNULL (MySQL)

```
SELECT Name, IFNULL(Email, 'No Email') AS Email
FROM Employees;
```

COALESCE (ANSI SQL)

```
SELECT Name, COALESCE(Email, Phone, 'No Contact') AS Contact
FROM Employees;
```

Result Example:

Name	Email	Phone	Email (IFNULL)	Contact (COALESCE)
Alice	alice@abc.com	9876543210	alice@abc.com	alice@abc.com
Bob	NULL	9123456780	No Email	9123456780
Charlie	NULL	NULL	No Email	No Contact

4 NULL Handling

- NULL means unknown or missing value.
- Comparisons with NULL using = or < > return UNKNOWN.

✗ Wrong

```
SELECT * FROM Employees
WHERE Email = NULL;
```

→ Returns no rows.

✓ Correct

```
SELECT * FROM Employees
WHERE Email IS NULL;
```

5 Conditional Logic in Queries

Use CASE to apply conditions in filtering, sorting, grouping or calculations.

```
SELECT Name, Salary,
  CASE
    WHEN Salary >= 70000 THEN 'High'
    WHEN Salary >= 50000 THEN 'Medium'
    ELSE 'Low'
  END AS SalaryLevel
FROM Employees
WHERE Department = 'IT'
ORDER BY Salary DESC;
```

CASE can be used in SELECT, WHERE, ORDER BY, GROUP BY, HAVING clauses.

Pro Tip

- ✓ Use COALESCE when you have multiple columns to check for NULL.
- ✓ Prefer IS NULL / IS NOT NULL instead of = NULL / < > NULL.

Interview Fact

CASE and COALESCE are frequently asked in SQL interviews to test real-world problem solving with conditional logic and NULL values.

DDL Commands

DDL (Data Definition Language) commands are used to define, modify and manage database objects.

1 CREATE

Used to create database objects like tables, views, indexes, etc.

Syntax:

```
CREATE TABLE table_name (
  column1 datatype constraints,
  column2 datatype constraints,
  ...
);
```

Example:

```
CREATE TABLE Employees (
  EmpID INT PRIMARY KEY,
  Name VARCHAR(50) NOT NULL,
  Salary DECIMAL(10,2)
);
```

2 ALTER

Used to modify the structure of an existing object.

Syntax:

```
ALTER TABLE table_name ADD column_name datatype;
ALTER TABLE table_name MODIFY column_name datatype;
ALTER TABLE table_name DROP COLUMN column_name;
ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
```

Example:

```
ALTER TABLE Employees ADD Email VARCHAR(100);
ALTER TABLE Employees MODIFY Salary DECIMAL(12,2);
ALTER TABLE Employees DROP COLUMN Email;
ALTER TABLE Employees RENAME COLUMN Name TO FullName;
```

3 DROP

Used to remove an existing database object permanently.

Syntax:

```
DROP TABLE table_name;
DROP VIEW view_name;
DROP INDEX index_name;
DROP DATABASE database_name;
```

Example:

```
DROP TABLE Employees;
DROP VIEW EmployeeSummary;
DROP INDEX idx_emp_name;
DROP DATABASE CompanyDB;
```

4 TRUNCATE

Used to remove all rows from a table, but keeps the table structure intact.

Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE Employees;
```

★ TRUNCATE is faster than DELETE and cannot be rolled back in some DBMS.

5 RENAME

Used to rename an existing database object.

Syntax:

```
ALTER TABLE old_table_name RENAME TO new_table_name;
ALTER VIEW old_view_name RENAME TO new_view_name;
```

Example:

```
ALTER TABLE Employees RENAME TO Staff;
ALTER VIEW EmpView RENAME TO StaffView;
```

★ RENAME syntax can vary slightly across different database systems.

Pro Tip

- ✓ Always backup before using DROP or TRUNCATE.
- ✓ ALTER is flexible but use carefully in production.
- ✓ Prefer TRUNCATE over DELETE when you need to remove all rows.
- ✓ Understand object dependencies before dropping tables or views.

Interview Fact

DDL commands cause an implicit COMMIT in most database systems.

DML & TCL Commands

DML (Data Manipulation Language) is used to manipulate data.
TCL (Transaction Control Language) is used to manage transactions.

1 INSERT

Used to insert new records into a table.

Purpose	Syntax	Example	Notes
Adds new rows to a table.	`INSERT INTO table_name (column1, column2, ...) VALLES (value1, value2, ...);`	`INSERT INTO Employees (EmpID, Name, Salary, DeptID) VALLES (101, 'Alice', 60000, 1);`	✓ Column list is optional if all columns are provided. ✓ Use NULL for missing values.

2 UPDATE

Used to modify existing records in a table.

Purpose	Syntax	Example	Notes
Updates data in existing rows.	`UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;`	`UPDATE Employees SET Salary = 65000 WHERE EmpID = 101;`	✓ Always use WHERE to avoid updating all rows. ✓ Supports multiple columns.

3 DELETE

Used to remove existing records from a table.

Purpose	Syntax	Example	Notes
Deletes rows that match a condition.	`DELETE FROM table_name WHERE condition;`	`DELETE FROM Employees WHERE EmpID = 101;`	✓ Without WHERE, all rows will be deleted. ✓ Cannot be rolled back after COMMIT.

4 COMMIT

Used to save all changes made in the current transaction.

Purpose	Syntax	Example	Notes
Permanently saves changes to the database.	`COMMIT;`	`COMMIT;`	✓ Ends the current transaction. ✓ Changes become permanent.

5 ROLLBACK

Used to undo changes made in the current transaction.

Purpose	Syntax	Example	Notes
Undoes changes that are not committed.	`ROLLBACK;`	`ROLLBACK;`	✓ Reverts all changes since the last COMMIT. ✓ Keeps database consistent.

6 SAVEPOINT

Used to set a point within a transaction.

Purpose	Syntax	Example	Notes

Allows partial rollback to a specific point.	`SAVEPOINT savepoint_name;`
	`-- do some operations`
	`ROLLBACK TO savepoint_name;`
	`RELEASE SAVEPOINT savepoint_name;`
		`SAVEPOINT sp1;`
		`UPDATE Employees SET Salary = 70000 WHERE EmpID = 101;`
		`ROLLBACK TO sp1;`
		`RELEASE SAVEPOINT sp1;`
			✓ Useful for complex transactions.
			✓ RELEASE SAVEPOINT is optional.

Pro Tip

- ✓ Always test with small datasets first.
- ✓ Use COMMIT only when you are sure.
- ✓ SAVEPOINT helps in safe step-by-step operations.

Interview Fact

DML affects data in tables.
TCL does not affect data, it controls transactions.

Constraints & Indexes

- Constraints enforce rules on data to maintain accuracy and integrity.
- Indexes improve the performance of data retrieval operations.

#	Constraint / Index	Purpose	Syntax (Column Level)	Example
1	PRIMARY KEY	Uniquely identifies each row in a table. No NULLs allowed. Only one per table.	column_name DATA_TYPE PRIMARY KEY	EmpID INT PRIMARY KEY
2	FOREIGN KEY	Maintains referential integrity between two tables.	column_name DATA_TYPE REFERENCES table_name(column_name)	DeptID INT REFERENCES Departments(DeptID)
3	UNIQUE	Ensures all values in a column (or set of columns) are unique. Allows one NULL.	column_name DATA_TYPE UNIQUE	Email VARCHAR(100) UNIQUE
4	CHECK	Ensures values satisfy a specific condition.	column_name DATA_TYPE CHECK (condition)	Salary DECIMAL(10,2) CHECK (Salary >= 0)
5	DEFAULT	Sets a default value when no value is supplied.	column_name DATA_TYPE DEFAULT default_value	Status VARCHAR(20) DEFAULT 'Active'
6	NOT NULL	Ensures a column cannot have NULL values.	column_name DATA_TYPE NOT NULL	Name VARCHAR(100) NOT NULL
7	INDEX	Improves the speed of data retrieval operations (SELECT queries).	CREATE INDEX index_name ON table_name(column_name);	CREATE INDEX idx_emp_name ON Employees(Name);
8	AUTO_INCREMENT / IDENTITY	Automatically generates a unique number for each new row.	MySQL : column_name INT AUTO_INCREMENT PRIMARY KEY SQL Server : column_name INT IDENTITY(1,1) PRIMARY KEY	MySQL : EmpID INT AUTO_INCREMENT PRIMARY KEY SQL Server : EmpID INT IDENTITY(1,1) PRIMARY KEY

Notes

- ✓ PRIMARY KEY = UNIQUE + NOT NULL.
- ✓ FOREIGN KEY prevents orphan records.
- ✓ Use INDEX on columns used in WHERE, JOIN, ORDER BY.
- ✓ Too many indexes can slow down INSERT/UPDATE/DELETE.

Interview Fact

- Constraints ensure data quality.
- Indexes improve performance.
- Both are crucial for real-world database design.

Pro Tip

- ✓ Always choose meaningful PRIMARY KEYS.
- ✓ Index foreign keys for better JOIN performance.
- ✓ Use DEFAULT and CHECK to reduce application-side validation.

Views & Stored Objects

Stored objects are database objects that store logic, improve security, reusability and performance.

1 VIEWS

Virtual table based on a SELECT query. Data is not stored physically.

Syntax (ANSI / Generic)

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

- Use `DROP VIEW view_name;`

Example

```
CREATE VIEW v_emp_dept AS
SELECT e.EmpID, e.Name, d.DeptName
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
WHERE e.Status = 'Active';
```

Key Points

- Simplify complex queries
- Restrict data access
- Updateable if based on single table (DBMS rules)

2 STORED PROCEDURES

A set of SQL statements stored and executed as a single unit.

Syntax (ANSI / Generic)

```
CREATE PROCEDURE proc_name (IN param1 datatype,
    OUT param2 datatype, ...)
BEGIN
  -- SQL statements
END;
```

- Execute: `CALL proc_name(...);`

Example

```
CREATE PROCEDURE GetEmployeesByDept (IN p_dept_id INT
BEGIN
  SELECT EmpID, Name, Salary
  FROM Employees
  WHERE DeptID = p_dept_id;
END;
CALL GetEmployeesByDept(10);
```

Key Points

- Improve performance
- Reuse logic
- Support parameters
- Easier maintenance

3 FUNCTIONS

Returns a single value. Can be used in SELECT, WHERE, etc.

```
CREATE FUNCTION func_name (params)
RETURNS return_type
BEGIN
  -- logic
  RETURN value;
END;
```

- Use in query like a built-in function.

```
CREATE FUNCTION GetTotalSalary (p_emp_id INT)
RETURNS DECIMAL(10,2)
BEGIN
 DECLARE v_total DECIMAL(10,2);
 SELECT SUM(Salary) INTO v_total
 FROM Employees WHERE EmpID = p_emp_id;
 RETURN v_total;
END;
  SELECT GetTotalSalary(101);
```

Key Points

- Must return a value
- Deterministic functions are preferred
- Use in SELECT, WHERE, JOIN, ORDER BY

4 TRIGGERS

Automatically executes when a specified event occurs on a table.

```
CREATE TRIGGER trigger_name
BEFORE | AFTER INSERT | UPDATE | DELETE
ON table_name
FOR EACH ROW
BEGIN
  -- trigger logic
END;
```

```
CREATE TRIGGER trg_emp_audit
AFTER INSERT ON Employees
FOR EACH ROW
BEGIN
 INSERT INTO EmpAudit (EmpID, ActionDate, Action)
 VALUES (NEW.EmpID, CURRENT_TIMESTAMP, 'INSERT');
END;
```

Key Points

- Events: INSERT, UPDATE, DELETE
- Timing: BEFORE / AFTER
- Row level triggers (most common)

5 SEQUENCES

Generate unique numeric values. (Database specific)

```
Oracle / PostgreSQL:
CREATE SEQUENCE seq_name
START WITH 1 INCREMENT BY 1;

SQL Server:
CREATE SEQUENCE seq_name
AS INT START WITH 1 INCREMENT BY 1;

MySQL: (No native sequence)
Use AUTO_INCREMENT instead.
```

```
-- Oracle / PostgreSQL
CREATE SEQUENCE emp_seq
START WITH 1000 INCREMENT BY 1;
-- Use
INSERT INTO Employees (EmpID, Name)
VALUES (NEXTVAL('emp_seq'), 'Alice');
-- MySQL
CREATE TABLE Employees (
  EmpID INT AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(100)
);
```

Key Points

- Ensures unique numbers
- Useful for keys, order numbers, etc.
- AUTO_INCREMENT (MySQL) is column property

Pro Tip

- ✓ Use VIEWS for security and simplicity.
- ✓ Use STORED PROCEDURES for business logic and performance.
- ✓ Use FUNCTIONS for calculations inside queries.
- ✓ Use TRIGGERS for auditing and enforcing rules.
- ✓ Use SEQUENCES / AUTO_INCREMENT for unique IDs.

Interview Fact

Employers love questions on Triggers, Procedures and View vs Table differences!

17

Window Functions

Window functions perform calculations across a set of table rows that are related to the current row without collapsing them into a single row (unlike GROUP BY).

OVER() Clause Structure

```
function() OVER (
    [PARTITION BY column_list]
    [ORDER BY column_list]
)
```

Key Components

- OVER() : Defines the window (the set of rows) for the function.
- PARTITION BY : Splits rows into partitions (like GROUP BY, but does not collapse rows).
- ORDER BY : Defines the order of rows within each partition.

#	Function	Purpose	Syntax	Example	Explanation
1	ROW_NUMBER() 1 → 2 → 3 → ...	Assigns a unique sequential number to rows within the partition.	ROW_NUMBER() OVER ([PARTITION BY col] ORDER BY col)	SELECT EmpID, Name, Salary, ROW_NUMBER() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS rn FROM Employees;	Rows in each department are numbered starting from 1 based on Salary (highest first).
2	RANK() 1 → 2 → 2 → 4 → ...	Assigns rank to rows with gaps for ties.	RANK() OVER ([PARTITION BY col] ORDER BY col)	SELECT EmpID, Name, Salary, RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS rnk FROM Employees;	Tied values get same rank, next rank is skipped. Example: 1, 2, 2, 4
3	DENSE_RANK() 1 → 2 → 2 → 3 → ...	Assigns rank to rows without gaps for ties.	DENSE_RANK() OVER ([PARTITION BY col] ORDER BY col)	SELECT EmpID, Name, Salary, DENSE_RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS drnk FROM Employees;	Tied values get same rank, next rank is not skipped. Example: 1, 2, 2, 3
4	LEAD() Salary → LEAD(Salary) 50000 → 45000 45000 → 40000 40000 → NULL	Accesses data from a following row in the window.	LEAD(column, offset, default) OVER ([PARTITION BY col] ORDER BY col)	SELECT EmpID, Salary, LEAD(Salary, 1) OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS NextSal FROM Employees;	Returns Salary of the next row (1 row ahead). If not available, returns NULL (or default).
5	LAG() Salary ← LAG(Salary) 50000 ← NULL 45000 ← 50000 40000 ← 45000	Accesses data from a previous row in the window.	LAG(column, offset, default) OVER ([PARTITION BY col] ORDER BY col)	SELECT EmpID, Salary, LAG(Salary, 1) OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS PrevSal FROM Employees;	Returns Salary of the previous row (1 row behind). If not available, returns NULL (or default).

PARTITION BY Example

```
SELECT EmpID, Name, DeptID, Salary,
ROW_NUMBER() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS rn
FROM Employees;
```

Data is split into partitions by DeptID. Row numbering starts from 1 in each partition.

DeptID	Name	Salary	ROW_NUMBER()
10	Alice	70000	1
10	Bob	60000	2
20	Charlie	65000	1
20	David	50000	2

Key Points

- Window functions do not reduce rows (no GROUP BY).
- Use PARTITION BY to create independent groups.
- Use ORDER BY inside OVER() for meaningful results.
- Very useful for rankings, running totals, comparisons, and analytical reports.

Pro Tip

- ✓ Always use ORDER BY with window functions for deterministic results.
- ✓ RANK vs DENSE_RANK: RANK leaves gaps, DENSE_RANK does not.
- ✓ LEAD/LAG are great for trend analysis and comparisons.

Interview Fact

Window functions are frequently asked in SQL interviews, especially for ranking, top-N, running totals and gaps & islands problems.

@qa_insights

18

SQL Interview Cheat Sheet

1 SQL Execution Order

1. FROM
2. JOIN
3. WHERE
4. GROUP BY
5. HAVING
6. SELECT
7. DISTINCT
8. ORDER BY
9. LIMIT / TOP

Process in which SQL clauses are evaluated. Knowing this helps to write correct and optimized queries.

2 JOIN Comparison

JOIN Type	Returns	Example Use
INNER JOIN	Matching rows from both tables	Common matching data
LEFT JOIN (LEFT OUTER)	All rows from left + matches from right	Keep all left data even if no match
RIGHT JOIN (RIGHT OUTER)	All rows from right + matches from left	Keep all right data even if no match
FULL JOIN (FULL OUTER)	All rows when there is a match on either side	Get everything from both tables
CROSS JOIN	Cartesian product (all combinations)	Rarely used (large result set)

3 Clause Execution Order

1. FROM (and JOIN)
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. DISTINCT
7. ORDER BY
8. LIMIT / TOP

Note

Aliases in SELECT can be used in ORDER BY but not in WHERE.

4 Common Interview Questions

- What is the difference between WHERE and HAVING?
- Different types of JOINs with examples.
- What are Primary Key and Foreign Key?
- How does NULL behave in SQL?
- Difference between UNION and UNION ALL?
- What is the use of INDEX?
- What are Subqueries and how do they work?
- Explain ACID properties.
- Difference between DELETE, TRUNCATE and DROP.
- What is the execution order of SQL clauses?

5 Performance Tips

- ✓ Use indexes on WHERE, JOIN and ORDER BY columns.
- ✓ Avoid SELECT * – fetch only required columns.
- ✓ Use proper JOINs instead of subqueries when possible.
- ✓ Avoid functions on indexed columns in WHERE clause.
- ✓ Use LIMIT / TOP to restrict large result sets.
- ✓ Analyze Execution Plan (EXPLAIN PLAN).

6 SQL Command Classification

Category	Commands	Purpose
DDL (Data Definition Language)	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define or modify database objects
DML (Data Manipulation Language)	SELECT, INSERT, UPDATE, DELETE	Retrieve or manipulate data in tables
DCL (Data Control Language)	GRANT, REVOKE	Manage user permissions and access
TCL (Transaction Control Language)	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

7 Best Practices

- ✓ Follow consistent naming conventions.
- ✓ Normalize data to reduce redundancy.
- ✓ Use constraints to ensure data integrity.
- ✓ Write modular queries (using CTEs, Views).
- ✓ Comment your SQL for better readability.
- ✓ Test with small dataset first, then scale.

8 Important Differences

Topic	Difference	Key Point
WHERE vs HAVING	WHERE filters rows before grouping, HAVING filters groups after grouping.	HAVING used with GROUP BY.
UNION vs UNION ALL	UNION removes duplicates, UNION ALL keeps duplicates.	UNION is slower than UNION ALL.
DELETE vs TRUNCATE	DELETE is DML, row by row & can be rolled back. TRUNCATE is DDL, faster & cannot be rolled back.	TRUNCATE resets identity (DB specific).
DROP vs TRUNCATE	DROP removes table structure. TRUNCATE removes data only.	DROP is permanent.

9 Quick Syntax Reminders

- `SELECT col1, col2 FROM table_name WHERE condition ORDER BY col1 ASC;`
- `INSERT INTO table_name (col1, col2) VALUES (val1, val2);`
- `UPDATE table_name SET col = val WHERE condition;`
- `DELETE FROM table_name WHERE condition;`
- `GROUP BY col HAVING COUNT(*) > 1;`
- `CREATE INDEX idx_name ON table_name(col);`

Remember

Good SQL skills = Better queries, Better performance, Better career!